Scout on BIRD: what we tried and what we learned

Experiment report for discussion. Yutesh, November 6, 2026.

The short version

The question: Scout is a text-to-SQL system. Can better prompting, voting and engineering make it more accurate on the BIRD benchmark, and where do its mistakes really come from?

1. **Best large result:** Adaptive got **225 of 251 (89.6%)** on corrected BIRD questions 50–300; Original got **217 of 251 (86.5%)**. Adaptive also cost about 28% less. The accuracy gap isn't clear evidence: they differ on 22 questions, 15 to 7 (sign test $p \approx 0.13$).
2. **Voting isn't the main limit.** In 72 of 86 misses, no candidate answer was right, so no better vote could have fixed them. A perfect chooser would move Mixed only from 76 to 80 of 100.
3. **Weak supervision didn't help.** It scored one lower than a plain vote on both runs we could compare. On Mixed it estimated models at 91–93% accurate when they were really 69–76%, probably because they often agree on the same wrong answer.
4. **The answer key matters about as much as the method.** Re-scoring Mixed's SQL against the corrected key flipped 18 of 100 results. The corrected team had changed the key on 18 of the 20 misses where no model was right.
5. **More information in the prompt didn't help.** A SQLite command guide gave 24 of 35 against 25 without it, with 26% more cost and 42% more time. That run was stopped early.
6. **A deterministic SQL compiler is promising, but unproven.** With a model writing strict English and a program writing the SQL, it reached 24 of 30, level with the older flows on those same questions (23–24). It still needs a run on 50–300.
7. **Nothing here is a final score.** All BIRD development so far used questions 1–300. Questions 301–500 are untouched, for one clean test.

> **The idea worth discussing:** model agreement may be most useful as a *signal* rather than a way to pick answers. It could flag answers Scout shouldn't trust, and possibly flag benchmark answer keys that are wrong.

Verdict at a glance

What we tried	Verdict	Evidence
Exact, full-row scoring	**Essential**	Removed false wrong answers. The scorer had compared only the first 200 rows.
Corrected answer key	**Essential**	18 of 100 results flip between keys.
Adaptive (stop early when 2 agree)	**Helped**	About 28% cheaper and over twice as fast. Accuracy at least level.
Full paper flow (4 chains)	**Unclear**	47 vs 43 on 50–100, but 40 vs 42 on 1–50 (older design). Most fixed calls.
Weak supervision	**Didn't help**	46 vs 47, and 75 vs 76. Free to compute, though.

Mixed (3 models from 3 labs)	**Not proven**	76 of 100, equal to its best model alone. Most expensive flow, and no fair baseline yet.
SQLite refresher doc	**Didn't help**	0 fixed, 1 broken in 35 questions. Tokens nearly doubled. Now off.
English format and compiler	**Promising**	24 of 30. Rejected chains fell from 43% to 19% after two format changes.
Data beats empty in the vote	**Small help**	4 fixed, 1 broken in replay across saved runs.

How the tests were set up

Scout reads a question and hint, looks up tables and values in a schema catalog, asks a model for meaning and SQL, checks the SQL with code, runs it read-only, and votes across tries.

Part	Detail
Official key	BIRD Mini-Dev: 500 questions across 11 databases, with BIRD's own hints and answer SQL.
Corrected key	The same set's 498 unique questions, with annotation errors fixed in questions, hints, answer SQL and descriptions (Arcwise-Plat, UIUC Kang lab).
Scoring	The same rule as BIRD's: result rows compared exactly as a set, with columns in order. Run with our own scorer; BIRD's script hasn't been run yet.
Default model	GPT-OSS 120B through OpenRouter, with temperature 0.7 for SQL, so identical runs can differ.
Development and test	Questions 1–300 have been seen and studied. Misses in 301–500 haven't been opened, and that range is held back for a clean test.
Fairness rules	No answer-specific rules. Prompt examples use a separate retail database. The answer key is read only after Scout answers.

The flows

Flow	How it works	Calls per question	Origin
Original	One meaning, a plan, 3 SQL tries from one prompt, a retry round if they disagree, then a majority vote.	5–6	Our baseline
Full	4 chains, each rewriting the question its own way and then writing SQL its own way, with its own examples. Majority vote, no retry.	8	Ask Me Anything, adapted
Weak supervision	Votes weighted by accuracy learned only from agreement (the triplet method). Replayed after a run, with no new calls.	0 extra	From the paper's family
Adaptive	Runs 2 chains and stops if they agree on an answer with data. Otherwise runs the other 2. Rejected SQL gets one retry.	4–12	Our idea
Mixed	3 models from 3 labs, with styles and methods shuffled per question. Majority vote live; weak supervision replayed.	6	Our extension
English program	Adaptive, but the model writes a strict English sentence and a deterministic program compiles it to SQL.	4–12	Our idea

Results side by side

Compare scores only within a row. Each row is one set of questions, and the rows differ a lot in difficulty.

Questions and key	Original	Full	Full + WS	Adaptive	Mixed	Mixed + WS	English
Corrected 1–50 (older design)	42	40		41			
Corrected 1–30 (subset of above)	24	23		23			**24**
Corrected 50–100 (51 questions)	43	**47**	46	**47** (replay)			
Corrected 101–200 (100)	**92**			90			
Corrected 200–300 (101)	82			**88**			
Corrected 50–300 (251)	**217**			**225**			
Official 1–100	no fair run				**76**	75	
Practice set, 20 (tuned on)	**20**				19	19	

English's 24 is after the two format changes; its first run got 22. Replaying the data-beats-empty rule gives 25, with no new run. The 251 total counts question 200 once. A 95% range on 76/100 is roughly 67–83, and on 24/30 roughly 63–90, so small samples can't separate these flows.

Experiment by experiment

1. Fixing how we measure

Tried: read wrong answers one by one before trusting any score.
Found: the scorer compared only the first 200 rows, which marked a right answer wrong (#1506). A crash on the apostrophe in `'Women's Soccer'` stopped a run at question 32, and the value check rejected correct doubled apostrophes. Rounded comparison was also replaced by BIRD's exact rule, which changed none of 442 saved right answers.
Takeaway: measurement bugs can look like model failures, so we fixed these first.

2. Official key vs corrected key

Tried: scored the same saved SQL against both keys, with no new model calls.
Result: Mixed goes from 76 on official to 78 on corrected, but that hides 18 flips: 10 misses become right and 8 rights become wrong. Scout read the official wording, and 42 of these questions have different wording or hints in the corrected version, so 78 isn't a true corrected score.
Takeaway: small score differences can be smaller than the key's own disagreement.

3. Ask Me Anything: Full and weak supervision

Tried: 3 SQL tries from one prompt often agreed and were wrong (6 of 8 misses on corrected 1–50 had high confidence), so we built 4 chains that ask in different ways, as the paper does. We adapted the idea to SQL by treating chains that return the same rows as one answer.
Result: Full 47 vs Original 43 on 50–100, but 40 vs 42 on an older design for 1–50. Weak supervision got 46, one below the plain vote.
Takeaway: different prompts on the same model gave some diversity, but learned weights had little to work with, because the chains' estimated accuracies were close.

4. Adaptive: stop when two chains agree

Tried: our idea to cut Full's cost. Two agreeing chains with data stop at 4 calls; otherwise all 4 run.
Result: 225 vs Original's 217 of 251, at about 4.3 calls vs 5.1. Won on 200–300 (88 vs 82), lost on 101–200 (90 vs 92), and tied Full on 50–100 by replay.
Takeaway: the clearest practical win is cost and speed. On 101–300, it used about 15% fewer calls, and cost 28% less ($0.24 vs $0.34).

5. Mixed: 3 models from 3 labs

Tried: if mistakes come from sharing one model, use different ones. GPT-OSS 120B, DeepSeek V4.1 Flash and Nemotron 3.5 Super, with styles and methods shuffled per question. A fourth, GLM 5.3 Flash, was removed after one question took 460 seconds, 131,072 tokens and $0.07.
Result on official 1–100: Mixed 76, weak supervision 75; DeepSeek alone 76, GPT-OSS 71, Nemotron 69. All 3 were right on 62 questions and none on 20. The first pass at a baseline, older Original SQL scored against the official key, gave 69, but that isn't fair: those runs read the corrected wording, and 7 of Mixed's 11 wins there may come from wording or key differences.
Takeaway: different labs didn't remove shared mistakes, and the vote never beat the best single model here.

6. SQLite refresher doc

Tried: I noticed Scout's SQL used fewer commands than BIRD's, so we added a guide to every SQL-writing call: about 60 examples and 3,700 tokens. Mixed reran on official questions, and we stopped it at 35.
Result: 24 vs 25; 0 fixed, 1 broken. Commands barely changed: `CAST`, `IIF` and ranking still never appeared, and `LEFT JOIN` stayed at 1 use. Tokens per question went from 13,399 to 25,686.
Limit: only 1 of the 10 misses that both runs shared here looked fixable, so this was a weak test.
Takeaway: models seem to know the commands. The harder part is choosing the right one.

7. English format and compiler

Tried: `english2sqlite.py` compiles a strict English format into SQL with no AI. It rejects anything ambiguous instead of guessing.

Test	Who wrote the sentences	Result
80 tests of 71 questions Scout had missed	Claude, after having seen many keys: a best case	All 80 compiled and ran. 54 matched the key, and another 10 official misses matched the corrected key.
Corrected 1–30, first run	GPT-OSS 120B	22 of 30. 40 of 94 chains rejected, even after a retry.
Corrected 1–30, after 2 format changes	GPT-OSS 120B	24 of 30. 17 of 90 chains failed. Calls 7.8 to 6.8 per question.

The 2 format changes: totals can carry their own condition or source, as in `(count of rows where c)`, and division now always returns a decimal, which fixed #1471's 0.
Takeaway: we found no translation mistake by the program in the misses reviewed. The remaining errors shift to reading the question and data, which the program can't fix, and the same 5 questions fail in every flow on this range. Rejecting ambiguity is also a real cost: it adds retries and sometimes leaves no answer.

8. Data beats empty in the vote

Tried: an empty answer now wins only if no chain found data, and still lowers confidence.
Result: replay across saved runs with votes fixed 4 and broke 1. The one broken is #1526 on the official key, whose right answer is itself empty; the rule fixes #1526 on the corrected key.
Takeaway: a small, free gain, and another sign of how much the keys differ.

Where the mistakes come from

Mixed's 24 misses on official 1–100



Blue: no model right, and the corrected team later changed the key. Orange: a model was right but lost the vote. Grey: no model right, and the key stayed the same (#1498, #1149).

Changing the key doesn't prove Scout was right. Of those 18, 10 become right under the corrected key, while the other 8 stay wrong, with changed wording or hints that make them hard to judge.

Was a right answer there to pick?

Flow and range	Misses	Right option present
Original, corrected 50–300	35	3
Full, corrected 50–100	4	2
Adaptive, corrected 101–300	23	5
Mixed, official 1–100	24	4
Total (some questions repeat across runs)	**86**	**14**

Adaptive may be undercounted: when it stops early, later chains never run.

16 questions where every option failed and the key wasn't changed

Here we read each question's saved meaning and SQL. Stage labels are my own reading, on a small sample chosen because every option failed.

Mistake	Stage	Count	Example
Ignored ties	SQL writing	3	#1032: `LIMIT 1` returned 1 of 3 tied leagues
Join lost rows	SQL writing	2	#788: plain `JOIN` dropped heroes with no publisher
Dropped part of the question	Understanding	1	#829: "which publisher" was gone from the meaning
Wrong level of total	Understanding	1	#1498: single record instead of a monthly total
Value format ignored	Understanding	1	#861: `0:01:54`, while data and hint use `1:54`
Percent left as a fraction	Both	1	#896: 0.54 instead of 54.2
Placeholder values counted	Data knowledge	1	#750: weight 0 most likely means unknown
Right data, strict scoring	Not a clear mistake	3	#898 column order; #931 number vs text

Doubtful key	Not a clear mistake	3	#791: the hint says all heroes, the key skips zeros

So of the 10 clear mistakes, 5 were at SQL writing, 3 at understanding, 1 from data knowledge and 1 across both steps. In the SQL-writing cases, the SQL did what the meaning said, but neither step considered how the data behaves. Most could be caught by code checks.

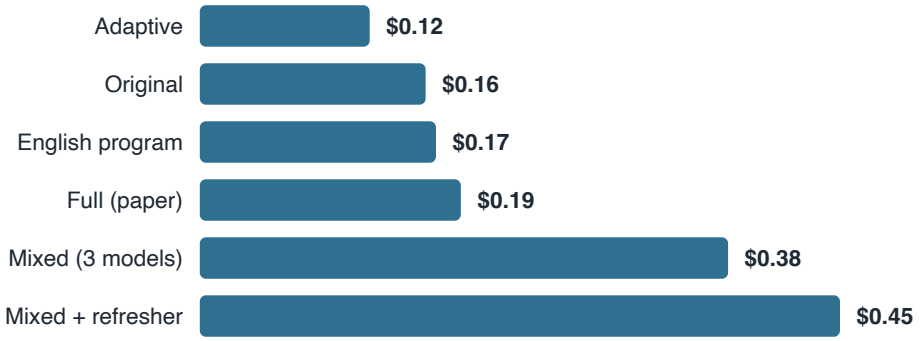
Were Scout's commands too limited?

Only partly. Questions using each command:

Command	Key, official 1–100	Mixed	Key, corrected 101–300	Original
`IIF`	11	0	2	0
`CAST`	16	2	24	10
`HAVING`	3	3	8	4
Subquery	6	22	38	63
`WITH`	0	6	6	7
`LEFT JOIN`	0	1	2	3

`IIF` and `CAST` are mostly style, since `CASE` and `* 1.0` do the same job. Scout does use `LEFT JOIN`, just not on the 2 questions in the 200–300 runs that needed it. That points to choosing commands, not knowing them.

What was expensive and what was cheap



Measured cost per 100 questions. Runs used different questions, so read this as a rough guide. English comes from 30 questions only. Adaptive is from its live runs on 101–300.

Flow	Calls per question	Tokens per question	Seconds per question	Cost per 100
Adaptive, corrected 101–300	4.3	18,900	3.6	$0.12
Original, corrected 50–300	5.1	22,900	8.3	$0.16
English, corrected 1–30	6.8	20,000	9.6	$0.17
Full, corrected 50–100	8.0	27,900	6.4	$0.19
Mixed, official 1–100	6.0	19,700	26.6	$0.38
Mixed + refresher, official 1–35	6.0	25,700	44.4	$0.45

Overhead	What it cost	Was it worth it?
Mixed's other models	About 3 times Adaptive's cost and 7 times its time per question	Not so far: equal to its best model
Refresher doc	On the same 35 questions: tokens +92%, cost +26%, time +42%	No. Turned off
Full's 4 chains on every question	8 fixed calls, about twice Adaptive's	Adaptive matched it on 50–100
English format guide	About 2,000 more tokens per sentence, plus retries (25 of 90 chains)	Unclear until 50–300 runs
GLM 5.3 Flash	One question: 131,072 tokens, 460 s, $0.07	No. Removed; no token limit added
Weak supervision	No model calls, but a lot of design and code	No gain so far

Cheap and useful: replaying saved answers cost nothing and gave some of the best findings: re-scoring on both keys, testing votes, finding whether a right answer was present, and counting commands. All these runs together cost only about $1.70: $1.38 billed in the export below, plus about $0.31 from later runs on Scout's own tracker. So model cost isn't the limit here. Time and careful evaluation are.

Checked against the real bill

The OpenRouter export has 3,859 calls and **$1.38**, from November 5 at 3:14 pm to November 6 at 1:38 am Central. It ends with the 76/100 Mixed run, so the refresher and English runs aren't in it; those figures come from Scout's own tracker.

Scout's cost tracker is accurate. Where a run's calls match the bill exactly, the amounts agree: Mixed $0.3749 tracked vs $0.3746 billed; Adaptive $0.1447 vs $0.1445; Adaptive $0.0967 vs $0.0965.

What each model cost in the 76/100 Mixed run

Model	Right alone	Billed	Median seconds per call	Slowest call	Output spent on thinking	Cache saving
GPT-OSS 120B	71	**$0.043**	1.6	6 s	72%	none
Nemotron 3.5 Super	69	$0.134	**10.2**	51 s	92%	almost none
DeepSeek V4.1 Flash	76	$0.198	1.6	33 s	87%	$0.096

- **GPT-OSS was by far the best value:** about a fifth of DeepSeek's cost for 5 fewer right answers.
- **Nemotron added the most overhead:** it scored lowest, cost 3 times GPT-OSS, and was the slowest. It's the likely reason Mixed took so long per question.
- **Hidden thinking is most of what output pays for.** Across the export, 70% of GPT-OSS's output tokens were reasoning, and it was 86–91% for DeepSeek and Nemotron.
- **Caching didn't reduce GPT-OSS's bill.** 80% of its prompt tokens were marked cached, but the bill still matches full list price ($0.883 billed vs $0.884). Output was 34% of its cost. So on GPT-OSS, a longer prompt like the refresher costs its full input price.

- **GLM's runaway was a single call:** 131,072 tokens, all of it thinking, 459 seconds, stopped by the length limit, and $0.066 of GLM's $0.083 total across 20 calls. No other call in the export hit a length limit.

What we observed overall

1. **Shared mistakes are the core problem.** Different prompts and labs still fail together. On Mixed's misses, 20 of 24 had no right option.
2. **Agreement is a signal, not proof.** "High confidence" was right 62 of 78 times (79%) on official BIRD, 211 of 239 (88%) for Original on corrected 50–300, and 18 of 18 on practice. It's most reliable on easy data.
3. **Hard ranges stay hard across flows.** On corrected 1–30, 5 questions (#1482, #1486, #1498, #1524, #1526) fail in every flow we ran there.
4. **Hints can be ignored or wrong.** #861's hint was in the prompt and ignored, while #791 follows its hint and the key disagrees.
5. **Knowing the data matters.** Reading "spent" as `Amount * Price`, where the key uses `Price`, failed on #1524 and #1526, and placeholder zeros changed #750.
6. **Rules can backfire on a benchmark.** Returning every tied row suits real users better, but BIRD's keys often use `LIMIT 1`, so it can lower the score.
7. **Practice scores flatter the system.** Practice is 95–100%, and official BIRD's first 100 is 76%.

How far to trust all this

> - **No clean test yet.** Rules were built while studying questions 1–300, so these scores may be optimistic.
> - **Small samples.** Many comparisons use 30–100 questions, and most gaps are within chance.
> - **Different question sets.** Mixed, on official, hasn't been compared fairly with Original or Adaptive on the same key.
> - **Older designs.** The corrected 1–50 runs predate some fixes, and saved no options.
> - **Few databases.** Official 1–100 covers 3 of 11 databases, and English 1–30 only one.
> - **One run each.** With temperature 0.7, we haven't measured how much identical runs differ.
> - **Human judgment.** Mistake types and stages are one reader's labels.
> - **Not BIRD's script yet.** Scoring follows its rule, but official numbers need its own script.

What I'd do next

#	Experiment	Why	Cost
1	**English on corrected 50–300**	The fair test against Adaptive's 225 of 251.	About $0.50
2	**Original on official 1–100**	A fair baseline for Mixed's 76.	About $0.16
3	**Wrong-key detector**	When all models agree against the key, how often is the key at fault? The corrected key gives a way to measure this.	Free

4	**Code checks**	Ties, lost join rows, empty results and percent scale. Measure wrong answers caught and right answers wrongly flagged on saved SQL.	Free to measure
5	**Scout says "not sure"**	Accuracy when Scout answers only when confident, using reworded questions and agreement.	Low to medium
6	**Different-model reviewer**	Compare a reviewer given only the result, given reasoning, and given assumptions plus data checks.	Medium
7	**Clean test on 301–500**	The one run that gives a score to report. Freeze the design first.	Low

Questions for the professor

1. **Annotation errors:** can model agreement detect wrong answer keys? Does prior work already do this, and how close is it to the annotation-error paper?
2. **Reporting:** when keys are noisy, what's the right way to report a score? Both keys, with the flips shown?
3. **Weak supervision:** our triplet estimates ran high, possibly because models share mistakes. How should we model that dependency when answers are open-ended and there are only 3–4 voters?
4. **Diversity:** labs and prompts didn't remove shared mistakes. What would produce real independence: different context, different schema views, or different data access?
5. **Abstaining:** is accuracy against the share of questions answered a better measure for enterprise text-to-SQL than plain accuracy? Is that where BEAVER points?
6. **Deterministic compilation:** is a strict intermediate language a research direction, or known semantic-layer work? What would make it new?
7. **Protocol:** is one clean run on 200 questions enough to claim anything, given how much 1–300 shaped the system?

Reference notes

Paper	How we used it
Arora, Narayan, … Sala, Ré. *Ask Me Anything: A simple strategy for prompting language models* (arXiv 2210.02441)	Read in full. The basis for Full.
Fu, Chen, Sala et al., 2020. *Fast and Three-rious*	The triplet method we built. Not read in full.
Varma, Sala et al., 2019. *Learning Dependency Structures for Weak Supervision Models*	Mentioned for copied mistakes. Not built, and not read in full.
The annotation-errors paper and BEAVER, suggested by the professor	The corrected BIRD key comes from the first. Exact citations still to be confirmed.

Paper titles for Fu and Varma are from memory, so check them before quoting. Run IDs: Original `214858-16cb`, `220637-d9d2`, `230005-0347`; Full `215521-f636`; Adaptive `222058-2206`, `231438-0cc0`; Mixed `005357-6721`; refresher `172022-2ef4`; English `182527-d798`, `184519-9578`. Not used: a stopped Mixed run on corrected 1–12 (10 of 12).